

TTFL: Trust-Flow-Driven Trusted Federated Learning for Edge Computing

Jiakai Zhou, Chao Bu, Shuo Zhang, Xinyu Liu
School of Computer Science and Engineering
Tianjin University of Technology, Tianjin, China
Email: wllsp0@stud.tjut.edu.cn, bc_0722@163.com

Abstract—Federated learning must distinguish a client’s past contribution from the risk of its current update. Reversible restrictions can nevertheless become permanent in practice if recovery requires an eligible peer population that is itself empty. Trust Flow uses separate historical-utility and risk states, a server-supported recovery audit, and layer gates scaled to an attainable score envelope. The controller preserves quarantine exclusion while allowing recovery evidence to advance without eligible peers. Parameter blocks receive sample-size-weighted, clipped updates only after current-state and score checks. An executable reference controller is evaluated with deterministic boundary cases and bounded randomized inputs. In a constructed test with all twenty clients initially quarantined, nonzero aggregation resumes at the fourth successful recovery audit and all clients return to NORMAL at the sixth. A maximal-risk sequence reaches BLACKLIST at submission eighteen, consistent with the specified EMA and persistence rules. These results establish the tested control behavior; classification robustness and hardware attestation performance remain outside this verification scope.

Index Terms—Federated learning, edge computing, trusted execution environment, risk modeling, layered aggregation, backdoor defense

1. Introduction

FL enables edge devices to train a shared model through parameter exchange while keeping their raw samples local [1], [2]. In an open edge deployment, however, the server must assess both the execution that produced an update and its effect on the global model. This assessment becomes difficult under Non-IID data: a legitimate long-tail client may repeatedly deviate from a reference direction, whereas an attacker may build a favorable record before submitting a harmful update. A single reputation score makes these cases difficult to separate. Tightening its threshold can remove useful heterogeneous contributions, while allowing past performance to compensate for current anomalies can delay the response to an attack. The resulting challenge is to preserve evidence of long-term contribution without allowing that evidence to conceal short-term risk.

Existing defenses use geometric, statistical, reference-based, or temporal evidence to flag unusual updates [3], [4],

[5], [6], [7]. Backdoor-oriented methods additionally examine model behavior, learned representations, clusters, and collusion [8], [9], [10], [11]. Such tests say little about how local training was executed, and the same tests can interpret Non-IID drift as an attack. TEE-assisted FL contributes a hardware-rooted execution boundary [12], [13], [14], [15]; admission based on that boundary still leaves the semantic safety of an admitted update unresolved.

The proposed *Trust Flow* policy addresses this tension by maintaining historical utility and short-term risk as separate states. Historical utility records contribution, while current risk can restrict an update despite a favorable earlier record. These states connect TEE-based execution admission, content auditing, and layer-wise aggregation. Their separation permits different responses to weak contribution and persistent risk, but does not by itself identify the cause of a deviation. The contributions are:

- 1) A recovery audit uses server evidence independently of eligible peers, removing the circular dependency between quarantine exit and peer-reference availability.
- 2) A layer gate is scaled to the best attainable score at each client’s evidence age, so uncertainty initialization cannot make every admissible score smaller than a fixed gate. State exclusion and an empty-survivor no-update rule remain explicit.
- 3) An executable controller and event-level verification connect evidence, transition counters, eligibility, numerical weights, and applied updates under constructed boundary conditions.

2. Background and Related Work

Federated learning in edge computing. Local training and server aggregation allow clients to collaborate without transferring raw samples, but leave the server dependent on the evidence accompanying each update. Heterogeneous data distributions complicate that assessment because a useful long-tail contribution may differ from the majority. Execution provenance and update quality consequently serve different purposes: the former establishes how an update was produced, while the latter determines how it should contribute to learning.

Poisoning and backdoor attacks. Sign flipping and gradient scaling disrupt optimization by changing update directions or magnitudes. Trigger, clean-label, and semantic backdoors instead seek targeted errors while preserving plausible main-task behavior, so their updates can remain close to an expected direction [16], [17]. Across rounds, an attacker may also accumulate a favorable history before injecting a harmful update, making current behavior and past contribution potentially conflicting sources of evidence.

Active defenses and their limits. Krum and Trimmed Mean remove geometric outliers [3], [4], FLTrust weights updates against a clean direction [5], and FoolsGold and FLDetector use cross-client or cross-round evidence [6], [7]. DeepSight, FLAME, CrowdGuard, RoseAgg, FLPurifier, ShieldFL, and RaSA inspect representations, backdoor behavior, collusion, or privacy-preserving updates [8], [9], [10], [11], [18], [19], [20]. TMT-FL and related systems connect execution integrity to model training [15]. These approaches observe different evidence spaces. Trust Flow places an interface between them: attestation determines execution eligibility, privacy-aware checks expose bounded statistics, content probes inspect the current update, and the two temporal streams decide future privileges. One observation can therefore alter a layer weight, a recoverable state, or a persistent state only when the corresponding rule is satisfied. The present evaluation isolates controller behavior; it does not rank these defenses by classification accuracy or attack success.

TEE-assisted trusted training. TEEs protect execution and key material and have been used to support FL integrity and attack mitigation [12], [13], [14], [15], [21]. Attestation can bind a measured environment to a reporting key; the signed report must additionally bind freshness and the submitted update. TTFL treats this authenticated reporting interface as a system assumption and evaluates the downstream controller separately.

3. System Boundary and Evidence Interface

The server receives an update ΔW_k and an authenticated report from client k , and applies $W^{(t)} = W^{(t-1)} + \Delta W_k^{(t)}$. The local-data factor is $p_k = n_k / \sum_j n_j$, where n_k counts the client’s training records. If a shared pool is replicated across clients, each local copy is counted in n_k . Filtering and clipping change the optimization trajectory, so sample-size weighting does not establish equivalence to the unfiltered objective over unique images.

The threat model permits poisoning of labels, inputs, objectives, and updates before report signing. The server follows the specified policy, sees individual updates, and holds a trusted proxy unavailable for client modification. Cryptographic secure aggregation is therefore outside this protocol. The reporting boundary in Fig. 1 protects a device key and binds identity, round, nonce, model digest, and update digest. GPU memory and sample semantics are outside that boundary. Protection of a report does not make externally collected telemetry truthful. Trusted acquisition is an assumption on the reporting interface, not a property

demonstrated by the controller tests. No platform-specific attestation or hardware timing result is claimed here.

The executable core accepts a valid-admission flag, normalized runtime entropy $h \in [0, 1]$, trainable update blocks, a server reference increment, and measured proxy loss and activation changes. It does not implement a TEE or a complete training application. A block is one named, floating-point trainable parameter tensor in a fixed manifest. Running statistics and integer buffers are excluded from both this manifest and the controller’s output; a training adapter must handle them separately without treating them as scored learning updates.

4. Evidence and State Control

Each round freezes the peer population, audits current updates, advances the states, and then aggregates eligible parameter blocks, as in Fig. 2. Let $\mathcal{D}_t$ contain valid submissions from clients not previously blacklisted. Before processing any state transition, freeze

$$\mathcal{R}_t = \{k \in \mathcal{D}_t : s_k^{(t-1)} \in \{\text{NORMAL}, \text{SUSPECT}\}, r_k^{(t-1)} < \tau_s\}. \quad (1)$$

A quarantined client supplies recovery evidence but never enters this round’s peer statistics. Peer comparisons exclude the evaluated client itself.

4.1. Admission and Runtime Reliability

Let $M_k \in \{0, 1\}$ be the admission-verification result. Invalid admission excludes the update and freezes its temporal state. For valid nonzero submissions with complete server evidence, the runtime filter uses

$$R_k = \exp(\lambda h_k), \quad (2)$$

$$Z_k = w_a + (1 - w_a)(1 - h_k), \quad (3)$$

$$P_k^- = P_k + Q, \quad K_k = P_k^- / (P_k^- + R_k), \quad (4)$$

$$T'_k = T_k + K_k(Z_k - T_k), \quad (5)$$

$$P'_k = (1 - K_k)P_k^-, \quad (6)$$

$$\text{TrustScore}_k = M_k T'_k / (1 + P'_k). \quad (7)$$

The fixed controller parameters are $w_a = 0.7$, $\lambda = 1$, $Q = 0.01$, $T_0 = 1$, and $P_0 = 0.25$. Entropy increases measurement uncertainty and reduces the observation mean; it is not a label of maliciousness. Tests supply h directly, so their results do not validate a particular telemetry histogram, instruction-counter collector, or TEE deployment.

4.2. Server Audit and Optional Peer Evidence

The mandatory audit uses a server reference and proxy measurements. For a training adapter, the reference increment is one full-proxy cross-entropy SGD step, $g = -10^{-3} \nabla_W \mathcal{L}_{\text{proxy}}(W)$, without momentum or weight decay. The controller consumes the resulting vector. Deterministic tests supply an explicit reference vector instead of running a model optimizer.

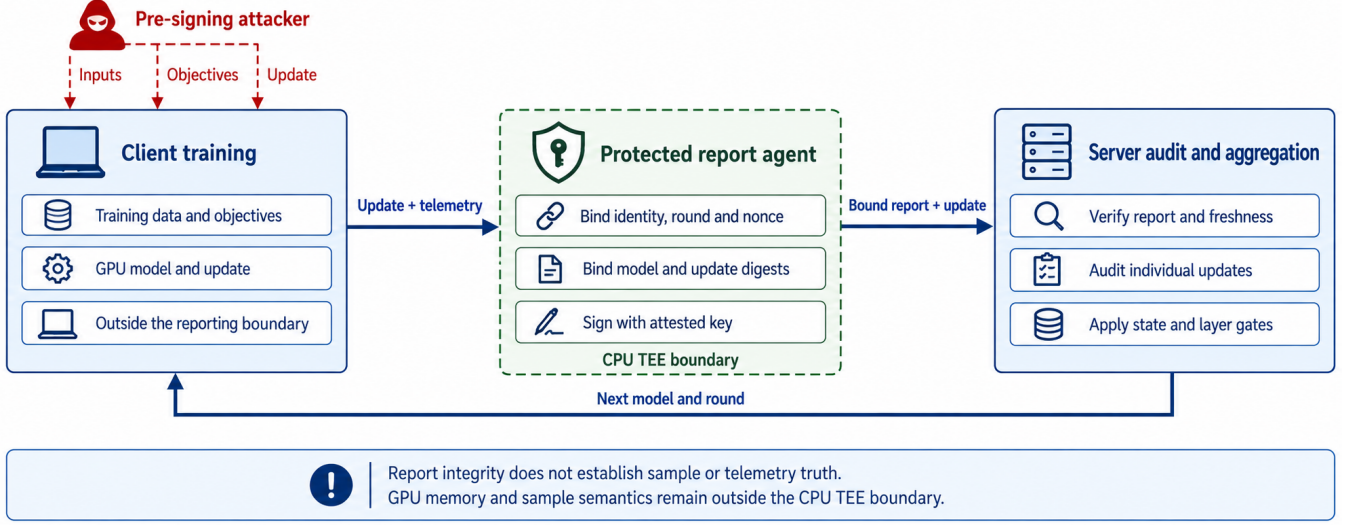


Figure 1. System roles and assumed reporting boundary. Red dashed paths locate attacks on inputs, objectives, or updates before signing. The protected agent binds identity, freshness, and model/update digests; the server audits individual updates before aggregation. This boundary does not protect GPU memory or establish sample and externally collected telemetry truth.

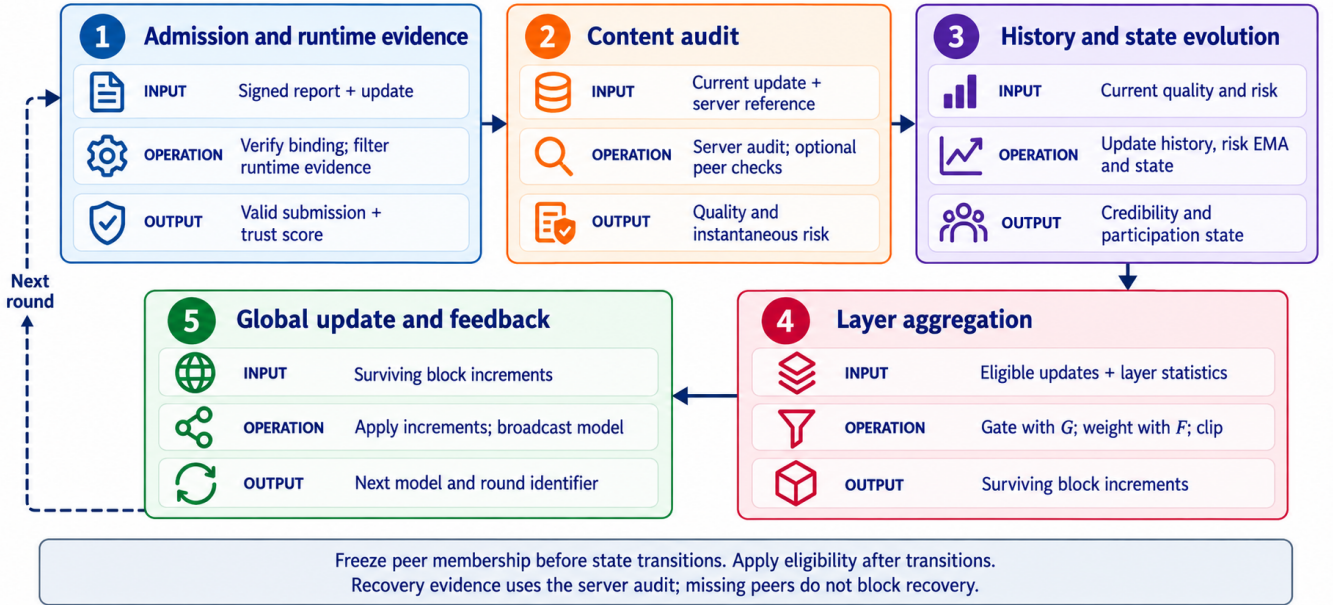


Figure 2. Five-stage trust-flow loop. The peer set is frozen before state transitions, while aggregation eligibility uses the updated state. A client starting in QUARANTINE may provide server-audited recovery evidence without entering that round’s peer statistics. Recovery to SUSPECT restores conditional aggregation eligibility in the same round; Table 1 specifies the transition rules.

A cosine is available only when both vector norms exceed $\varepsilon_n = 10^{-12}$. An unavailable cosine is omitted from risk fusion; it is not set to zero and interpreted as disagreement. Let c_k denote the available reference cosine. The direction risk is $(1 - c_k)/2$ and the reference quality is $v_k = 0.6 + 0.4c_k$. When the reference direction is unavailable, proxy behavior defines $v_k = (1 - r_{loss,k})(1 - r_{act,k})$. For usable peers $j \in \mathcal{R}_t \setminus \{k\}$, positive trust weights produce

a mean cosine $\bar{c}_k$ and $a_k = 0.6 + 0.4\bar{c}_k$. Then

$$q_k = \begin{cases} 5a_kv_k/(4a_k + v_k), & \text{usable peers exist,} \\ v_k, & \text{otherwise.} \end{cases} \quad (8)$$

All available quantities lie in $[0, 1]$; no peer value is imputed when the set is empty. The peer-risk channel, when available, is $r_{peer,k} = \text{clip}(1 - \bar{c}_k, 0, 1)$.

Write $\Delta\mathcal{L}_k = \mathcal{L}_{proxy}(W + \Delta W_k) - \mathcal{L}_{proxy}(W)$. The mandatory server channels are

$$r_{loss,k} = \text{clip}\left(\frac{\max\{0, \Delta\mathcal{L}_k\}}{\delta_{loss}}, 0, 1\right), \quad (9)$$

$$r_{act,k} = \text{clip}(d_{act,k}/\delta_{act}, 0, 1). \quad (10)$$

For a batch of B matched inputs and activation vectors A_i at a declared representation, activation drift is

$$d_{act,k} = \frac{1}{B} \sum_{i=1}^B \text{KL}(\text{softmax}(A_{base,i}) \parallel \text{softmax}(A_{k,i})). \quad (11)$$

Softmax is taken over flattened features of each sample, never over the batch. The same inputs and evaluation mode are used before and after the candidate update. For a ResNet-18 adapter, the declared representation is the 512-vector after global average pooling. The array implementation tests this sample axis; it does not claim a trained ResNet measurement.

Each scale uses trusted calibration observations of its own nonnegative quantity: $\delta = \max\{10^{-6}, \text{median}(z) + 3 \text{MAD}(z)\}$. Calibration inputs are separate from final evaluation inputs. Positive scales are required inputs to the controller. Recovery fixtures use synthetic zero loss/activation changes and scales of one; those are test inputs, not fitted CIFAR-10 calibration values.

Amplitude risk is $\text{clip}(\|\Delta W_k\|/b_k - 1, 0, 1)$, where b_k is the median nonzero norm of usable peer updates. With no usable peer, $b_k = \|g\|$. The channel is omitted if this scale is at most ε_n . Instantaneous risk z_k is the maximum of the two mandatory server channels and all available geometric channels. Missing proxy evidence freezes history, risk, and counters and yields zero aggregation weight. A full-update norm at most ε_n is a no-operation submission with the same freeze rule; it cannot build reputation or create a direction alarm.

This separates recovery from peer availability: when all clients are quarantined, valid nonzero updates can still be evaluated against the trusted server. Loss of the server audit service intentionally suspends recovery; that external dependency is distinct from a circular peer-state dependency.

4.3. History, Risk, and Persistent Transitions

Figure 3 presents the four-state participation design from the original study. The executable controller uses the explicit transition and persistence rules in Table 1.

Each complete nonzero submission adds unit evidence to the history model:

$$a'_k = \beta_h a_k + q_k, \quad b'_k = \beta_h b_k + 1 - q_k, \quad (12)$$

$$H'_k = a'_k / (a'_k + b'_k), \quad r'_k = \beta_r r_k + (1 - \beta_r) z_k. \quad (13)$$

Initialization is $a_0 = b_0 = 1$, $r_0 = 0$; $\beta_h = 0.9$ and $\beta_r = 0.85$. The strictly positive evidence masses require no additive denominator offset. History and risk share some geometric inputs and are not statistically independent evidence streams.

Utility threshold. The review condition is

$$B_k = (H'_k < 0.26) \vee (r'_k \geq 0.64). \quad (14)$$

Thus 0.26 is a lower bound on historical utility, not a threshold on $1 - H_k$. A favorable recovery observation satisfies $\neg B_k$. The transitions in Table 1 are evaluated using the starting state and updated history/risk. At most one transition occurs per complete submission; escalation has priority. Each counter counts consecutive qualifying observations in its starting state, resets on a failed condition, and all counters reset after a transition. Missing or no-operation submissions freeze counters. There is no instantaneous override of the EMA or extra attack-specific blacklist branch.

TABLE 1. CONTROLLER TRANSITIONS; ALL THRESHOLDS APPLY TO UPDATED EVIDENCE.

Start	Condition	End
NORMAL	B_k for 2 complete submissions	SUSPECT
SUSPECT	B_k for 3 submissions, or $z_k \geq 0.80$	QUARANTINE
QUARANTINE	$r'_k \geq 0.90$ for 4 submissions	BLACKLIST
QUARANTINE	$\neg B_k$ for 2 submissions	SUSPECT
SUSPECT	$\neg B_k$ for 2 submissions	NORMAL

Newly quarantined or blacklisted updates are excluded immediately. BLACKLIST also rejects future submissions. Recovery to SUSPECT restores conditional current-round eligibility. Recovery does not itself imply passing a layer gate or applying a nonzero model update; those endpoints are logged separately.

5. Aggregation with Attainable Gates

Figure 4 separates the evidence inputs, layer decisions, and constrained execution. The normalized score G_k determines gate passage, whereas the raw credibility F_k determines each survivor's relative weight.

Let $\mathcal{E}_t$ contain complete nonzero audited submissions. After state transitions, form $\mathcal{A}_t = \{k \in \mathcal{E}_t : s_k^{(t)} \in \{\text{NORMAL}, \text{SUSPECT}\}\}$. The unnormalized credibility score remains

$$F_k = \text{TrustScore}_k^3 q_k \sqrt{H'_k (1 - r'_k)}. \quad (15)$$

A fixed gate near one is unsuitable because filter uncertainty suppresses the entire attainable score range. Instead, normalize only the gate comparison by a deterministic best-evidence envelope at the client's evidence age n_k^e . Here n_k^e counts complete nonzero submissions and is distinct from the training-record count n_k .

Starting from $\underline{P}_0 = 0.25$, $a_0^+ = b_0^- = 1$, define

$$\underline{P}_n = \frac{\underline{P}_{n-1} + Q}{1 + \underline{P}_{n-1} + Q}, \quad (16)$$

$$a_n^+ = 0.9a_{n-1}^+ + 1, \quad b_n^- = 0.9b_{n-1}^-, \quad (17)$$

$$U_n = (1 + \underline{P}_n)^{-3} \sqrt{a_n^+ / (a_n^+ + b_n^-)}. \quad (18)$$

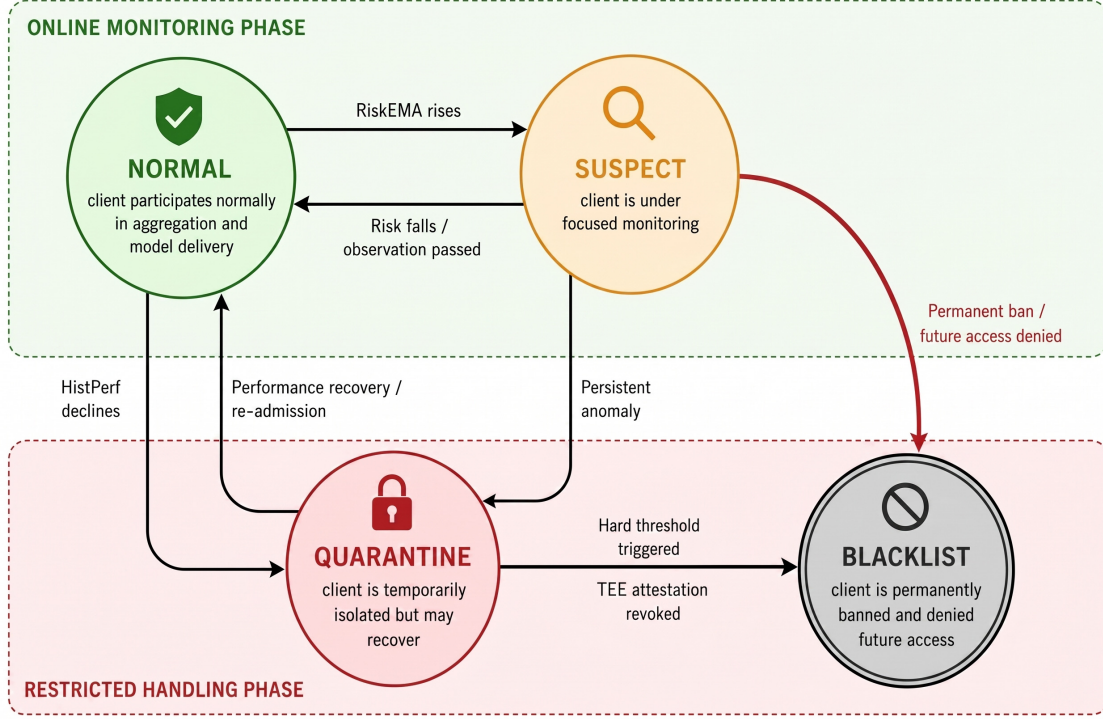


Figure 3. Original four-state participation design, with its original arrows and labels retained. The direct shortcut paths shown here belong to that design; Table 1 specifies the stepwise transitions used in the executable controller verification.

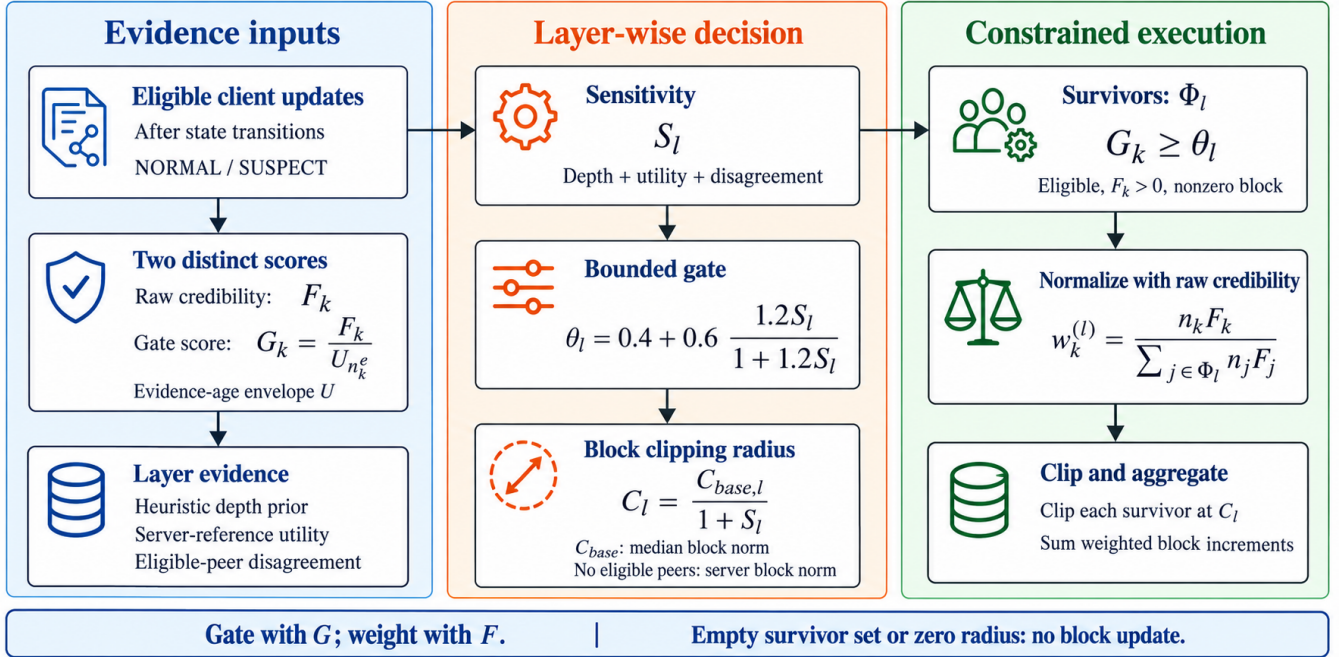


Figure 4. Layer-wise aggregation after state transitions. Only eligible NORMAL/SUSPECT submissions can survive a gate. The comparison uses $G_k = F_k / U_{n_k^e}$, but weights retain $n_k F_k$. Depth is a heuristic prior; clipping uses a separate scale for each parameter block. An empty survivor set or a radius at most ε_n produces no block update, without bypassing state exclusion.

Since $R \geq 1$, $T \leq 1$, $q \leq 1$, and $r \geq 0$, induction gives $F_k \leq U_{n_k^e}$. The bound is attained with valid admission,

$h = 0$, $q = 1$, and $z = 0$ at every complete observation. The dimensionless gate score is $G_k = F_k/U_{n_k}$; actual weights continue to use F_k , preserving the covariance penalty there.

5.1. Layer Statistics and Clipping

Layer statistics use $\mathcal{T}_t = \mathcal{R}_t \cap \mathcal{A}_t$. This is a current-eligibility filter on the frozen peer set. Newly recovered clients may aggregate but cannot influence the same-round layer statistics. For blocks $l = 1, \dots, L$, define

$$S_{depth}^{(l)} = e^{-3l/L}, \quad (19)$$

$$S_{util}^{(l)} = \|g^{(l)}\| / \max_m \|g^{(m)}\|, \quad (20)$$

$$S_{sec}^{(l)} = \frac{1}{2}(1 - \text{mean}_{k \in \mathcal{T}_t} \cos(\Delta W_k^{(l)}, g^{(l)})), \quad (21)$$

$$S_l = 0.3S_{depth}^{(l)} + 0.3(1 - S_{util}^{(l)}) + 0.4S_{sec}^{(l)}. \quad (22)$$

Only available cosines enter the mean. An empty cosine set gives no peer disagreement contribution, $S_{sec} = 0$, and is recorded as missing peer evidence. If all root blocks have zero norm, set $S_{util} = 0$. The depth component is a heuristic prior, not a measured privacy risk or a privacy guarantee.

The bounded gate and clipping radius are

$$\theta_l = 0.4 + 0.6 \frac{1.2S_l}{1 + 1.2S_l}, \quad C_l = \frac{C_{base,l}}{1 + S_l}. \quad (23)$$

Here $C_{base,l}$ is the median norm of block l over $\mathcal{T}_t$; it is not the norm of the complete model update. If $\mathcal{T}_t$ is empty, use the corresponding server-reference norm $\|g^{(l)}\|$. A radius at most ε_n leaves that block unchanged. Thus trusted server evidence also supplies a defined aggregation path after collective quarantine, without using quarantined updates to set their own thresholds or clipping radii.

Because $0 \leq S_l \leq 1$, $0.4 \leq \theta_l \leq 0.727273 < 1$. The attainable best-evidence score $G_k = 1$ passes every such gate, including at the first complete submission. This eliminates structural impossibility caused by the score envelope; it does not guarantee that noisy or heterogeneous updates pass.

5.2. Survivors and Applied Updates

For a positive clipping radius, survivors are

$$\Phi_l = \{k \in \mathcal{A}_t : G_k \geq \theta_l, F_k > 0, \|\Delta W_k^{(l)}\| > \varepsilon_n\}. \quad (24)$$

If Φ_l is empty, $\Delta W_{global}^{(l)} = 0$. No state exclusion or gate is bypassed by a fallback. Otherwise positive survivor mass permits exact normalization:

$$w_k^{(l)} = \frac{n_k F_k}{\sum_{j \in \Phi_l} n_j F_j}, \quad (25)$$

$$\widehat{\Delta W}_k^{(l)} = \Delta W_k^{(l)} \min\{1, C_l / \|\Delta W_k^{(l)}\|\}, \quad (26)$$

$$\Delta W_{global}^{(l)} = \sum_{k \in \Phi_l} w_k^{(l)} \widehat{\Delta W}_k^{(l)}. \quad (27)$$

Weights sum to one on a nonempty survivor set. Each clipped block norm is at most C_l , so the triangle inequality

also bounds the aggregate norm by C_l . Individual nonzero contributions can cancel; a selected block, a positive weight, and a nonzero aggregate increment are therefore distinct endpoints.

The audit records client, round, trainable-block name, starting/ending state, quality, instantaneous and smoothed risk, history, persistence counters, transition reason, raw and gate scores, threshold, survivor membership, normalized weight, pre/post-clipping norms, and applied contribution norm. Per-block logs additionally record survivor count, clipping radius, skipped updates, and the norm of the summed increment.

6. Original Training Study

The original CIFAR-10 study examines model learning, client-level behavior, component effects, and sensitivity to data heterogeneity. Its figures are reproduced below with their plotted values, event labels, and training horizons preserved. These results describe the original implementation; the constructed tests in Section 7 evaluate the revised executable controller separately. Restoring the figures does not constitute a new training run of that controller.

6.1. Learning and Client Behavior

Figure 5(a) shows the original accuracy trajectory across communication rounds and the ASR baseline as labeled in the original plot. Figure 5(b) shows that client historical utility and risk can evolve differently. Detection, interception, and quarantine retain their original event labels rather than being reinterpreted as permanent blacklist events. Figure 5(c) provides a qualitative feature-space view rather than a separability guarantee.

6.2. Component Analysis

Figure 6(a) compares the clean-reference, shallow-probe, and full-probe variants. Its untargeted attack categories and targeted backdoor category are not interpreted as a common targeted-ASR comparison. Figure 6(b) compares Global-Clipping, hierarchical gating without re-normalization, and the full method over the original 50-round horizon. Its round-30 annotation is retained. Because those variants also differ in their aggregation operations, their entire performance gap cannot be attributed to normalization alone.

6.3. Data-Heterogeneity Sensitivity

Figure 7 reproduces the original accuracy comparison across Dirichlet parameters. In that reported sweep, the Trust Flow series varies less than the Krum and FLTrust series. The plot characterizes the original data-partition setting; the Dirichlet parameter alone does not describe the full local distribution when clients also use a shared pool. No additional uncertainty estimates or shared-pool ablation results are inferred from the plotted points.

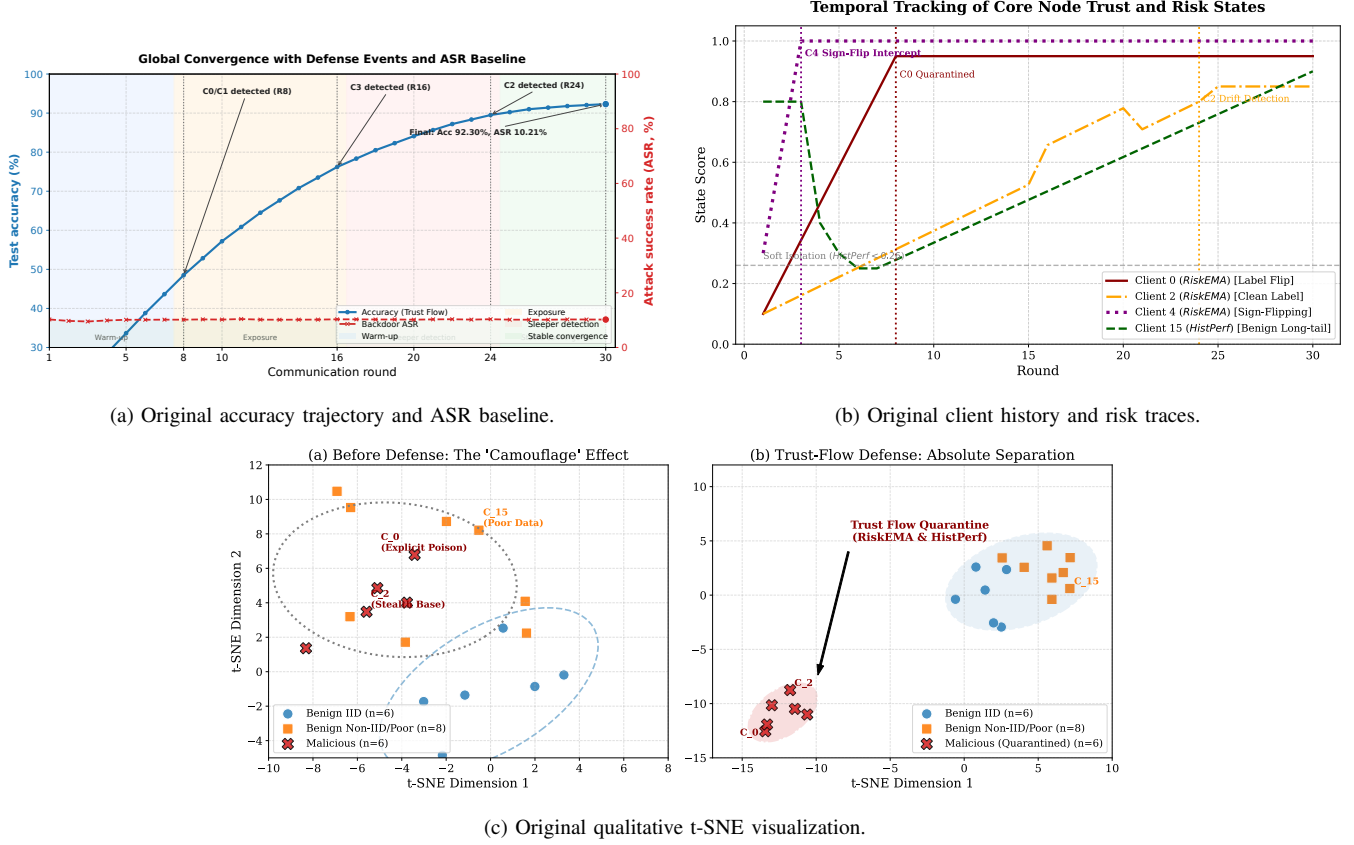


Figure 5. Original-study learning trajectory and ASR baseline (a), client history and risk (b), and qualitative t-SNE view (c).

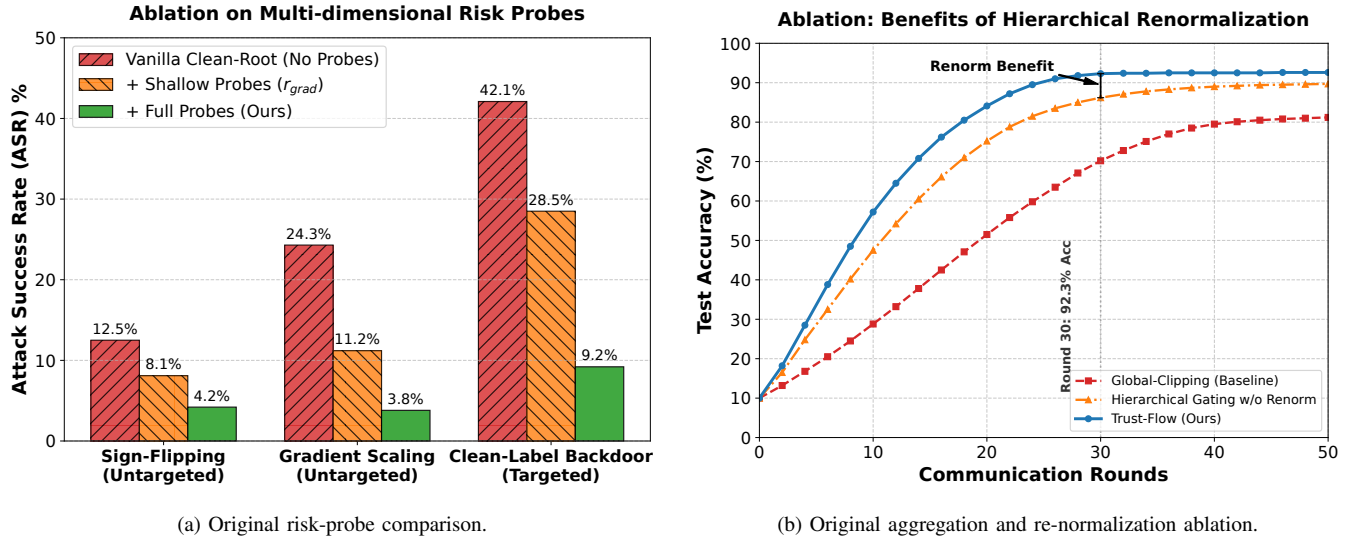


Figure 6. Original-study probe comparison (a) and aggregation ablation (b). Original axes and category labels are retained; panel (b) shows the complete 50-round horizon.

7. Executable Protocol Verification

7.1. Scope and Reproduction

The evidence in this section is generated by the reference controller and fixed configuration, not by training a classi-

fier. `protocol/controller.py` implements the audit, states, envelope, and block aggregation; `config.json` fixes its parameters. `validate.py` executes eleven test groups and exports state, block, and layer events plus SHA-256 hashes. Constructed observations are explicitly

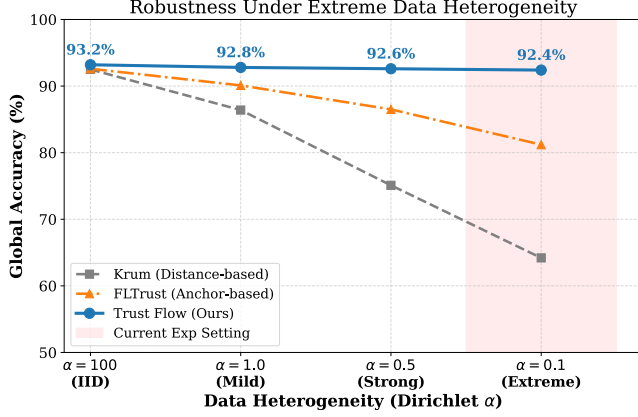


Figure 7. Accuracy under the data-heterogeneity settings reported in the original study. Original points, methods, and axis labels are retained.

identified as test inputs. The randomized check uses seed 20261001, 100 sequences, and 100 observations per sequence. No repeated-training SD or statistical superiority is inferred from these checks. Table 2 lists the fixed conditions and checked endpoints.

Section 6 reports the original training figures. The tests below establish specific executable properties of the revised controller and expose the conditions under which recovery is possible; they do not remeasure the original classification results.

7.2. Recovery and Startup Behavior

The previous utility-deficit interpretation triggered review whenever $H < 0.74$. With $q = 0.8$, $z = 0.3$, and the stated initial evidence, the first five history values are 0.6071, 0.6619, 0.6951, 0.7171, and 0.7328. The legacy two/three-step review policy enters collective quarantine at the fifth observation. When mandatory peer evidence is then unavailable, the history freezes. The independent reproduction retains that frozen value through round 40. Using the utility floor $H < 0.26$ removes this startup restriction for the same constructed sequence.

To test recovery independently of that threshold correction, a second test places all twenty clients directly in QUARANTINE with initial risk 0.95, initial history 0.5, and reset counters. Every subsequent submission is valid, nonzero, aligned with the server reference (1,0), and has zero proxy loss and activation change. Calibration scales are one and entropy is zero. Thus the recovery audit has $q = 1$, $z = 0$ even while $\mathcal{R}_t = \emptyset$. Risk falls below 0.64 at the third audit; the second favorable audit at round four permits QUARANTINE $\rightarrow$ SUSPECT. All clients receive positive weights and produce a nonzero aggregate in that round. They return to NORMAL at round six (Fig. 8 in the appendix). This is a conditional recovery result under favorable trusted-server evidence, not a guarantee for every benign data distribution.

7.3. Risk Timing and Gate Reachability

With $r_0 = 0$ and $z_t \leq 1$, pure EMA satisfies $r_t \leq 1 - 0.85^t$. At $t = 14$ its upper bound is 0.89723, below the blacklist threshold; the first crossing is at $t = 15$. Four consecutive qualifying observations while quarantined therefore cannot finish before $t = 18$. The maximal-risk test matches that bound and rejects the next submission at $t = 19$.

For the earlier unscaled gate $0.4 + 1.2S_t$, the best first-round raw score is only 0.469222. Together with the depth component, this excludes blocks with $l/L < 0.549594$ in the first round even under favorable other evidence. Its asymptotic raw-score ceiling is about 0.761393, so simply clipping gates to one would leave another unreachable region. These are analytical bounds, not measured gate frequencies. The revised normalized comparison attains one under ideal evidence at every age, while the maximum gate is 0.727273. The 10,100-case sweep verifies the executable formulas against this envelope. It establishes reachability, not an acceptable survivor rate on trained models.

8. Limitations and Conclusion

The controller removes a circular dependency in recovery and aligns layer gates with the score range produced by filter initialization. Its tests link persistent state transitions to numerical aggregation behavior, including empty gates, missing peers, zero updates, and sample-size weighting. They also distinguish reversible state restrictions from permanent bans and selected blocks from applied updates.

The verification does not measure clean accuracy, targeted attack success, benign recovery under real distribution shifts, shared-pool dependence, or hardware cost. Those claims require training runs whose configuration and logs match this protocol, disjoint proxy/calibration/final-evaluation records, and per-attack outcomes with explicit target-class exclusions. Trusted-server availability and calibration quality remain substantive assumptions. Separate history and risk states do not establish Byzantine robustness or superiority over a tuned single-state policy. The executable specification and event logs provide a concrete basis for those subsequent comparisons.

TABLE 2. EXECUTED CONTROL TESTS. INPUTS ARE CONSTRUCTED; NONE IS A CIFAR-10 OR HARDWARE MEASUREMENT.

Case	Fixed condition	Checked result
Legacy deadlock reproduction	$q = 0.8, z = 0.3$, utility cutoff 0.74, mandatory peers	SUSPECT at 2, QUARANTINE at 5; frozen at 6–40
Healthy startup	$q = 0.8, z = 0.3$, utility floor 0.26	NORMAL throughout 40 complete observations
Collective recovery	20 clients initially quarantined, $r_0 = 0.95$; server audit available	Nonzero aggregate at 4; all NORMAL at 6
Maximal risk	$q = 1, z = 1, r_0 = 0$, pure EMA	SUSPECT at 8; QUARANTINE at 9; BLACKLIST at 18
Gate envelope	Ideal evidence for 100 ages, 101 sensitivities	All 10,100 age-sensitivity checks pass
Empty survivors	Low-score candidate; high-score quarantined client	Zero increment, no backfill
Missing evidence / one client	Missing proxy; then valid server audit without peers	Freeze on missing proxy; sole survivor weight 1
Zero / tiny update	Norms 0, 10^{-15} , 10^{-12}	No direction alarm, history gain, or applied update
Sample weights / clipping	Equal scores, training counts in ratio 1 : 3	Weights 0.25/0.75; aggregate norm within radius
Calibration / activation	Zero calibration values; two-sample activations	Positive 10^{-6} floor; KL uses sample-wise softmax
Bounded random evidence	10,000 bounded observations	Bounded states and scores; excluded states have no eligibility

Appendix A. Constructed Recovery Trace

Figure 8 shows the fixed-input controller test described in Section 7. All clients submit the same two-dimensional update at every audit. The discrete state steps and constant applied-update norm after recovery follow from those test inputs; they are not classifier learning curves.

References

- [1] P. Kairouz, H. B. McMahan, B. Avent *et al.*, “Advances and open problems in federated learning,” *Foundations and Trends in Machine Learning*, vol. 14, no. 1–2, pp. 1–210, 2021.
- [2] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agueray Arcas, “Communication-efficient learning of deep networks from decentralized data,” in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, vol. 54, 2017, pp. 1273–1282. [Online]. Available: <https://proceedings.mlr.press/v54/mcmahan17a.html>
- [3] P. Blanchard, E. M. El Mhamdi, R. Guerraoui, and J. Stainer, “Machine learning with adversaries: Byzantine tolerant gradient descent,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017. [Online]. Available: <https://papers.nips.cc/paper/6617-machine-learning-with-adversaries-byzantine-tolerant-gradient-descent>
- [4] D. Yin, Y. Chen, K. Ramchandran, and P. Bartlett, “Byzantine-robust distributed learning: Towards optimal statistical rates,” in *Proceedings of the 35th International Conference on Machine Learning*, vol. 80, 2018, pp. 5650–5659. [Online]. Available: <https://proceedings.mlr.press/v80/yin18a.html>
- [5] X. Cao, M. Fang, J. Liu, and N. Z. Gong, “FLTrust: Byzantine-robust federated learning via trust bootstrapping,” in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2021.
- [6] C. Fung, C. J. M. Yoon, and I. Beschastnikh, “The limitations of federated learning in Sybil settings,” in *Proceedings of the 23rd International Symposium on Research in Attacks, Intrusions and Defenses (RAID)*, 2020, pp. 301–316. [Online]. Available: <https://www.usenix.org/conference/raid2020/presentation/fung>
- [7] Z. Zhang, X. Cao, J. Jia, and N. Z. Gong, “FLDetector: Defending federated learning against model poisoning attacks via detecting malicious clients,” in *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2022, pp. 2545–2555.

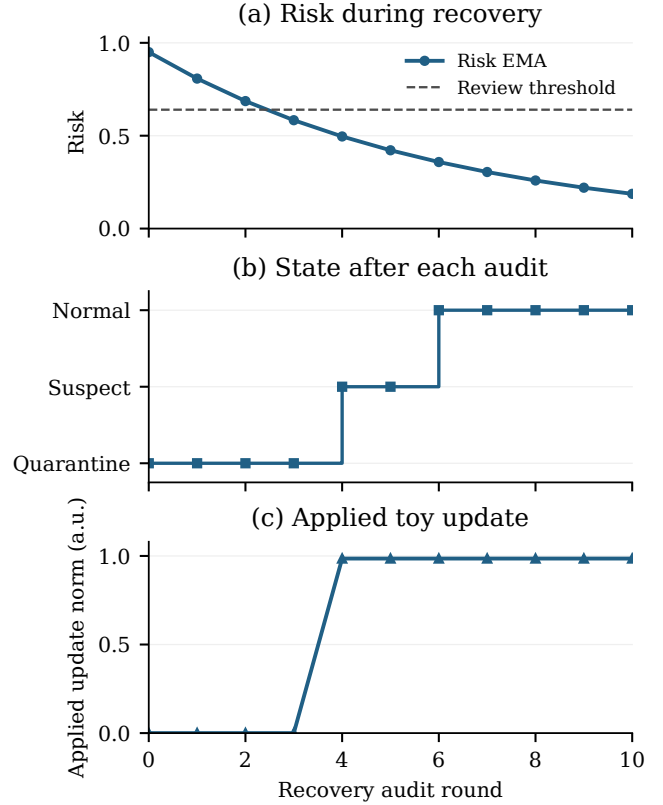


Figure 8. Constructed collective-quarantine recovery test. All twenty clients have identical inputs and states; one common state/risk trajectory is shown. The bottom panel is the applied aggregate increment of the two-dimensional test block, in arbitrary units. Peer statistics are empty when aggregation resumes at audit four.

- [8] P. Rieger, T. D. Nguyen, M. Miettinen, and A. R. Sadeghi, “Deep-Sight: Mitigating backdoor attacks in federated learning through deep model inspection,” in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2022.
- [9] T. D. Nguyen, P. Rieger, H. Chen, H. Yalame, H. Möllering, H. Fereidooni, S. Marchal, M. Miettinen, A. Mirhoseini, S. Zeitouni, F. Koushanfar, A.-R. Sadeghi, and T. Schneider, “FLAME: Taming

- backdoors in federated learning,” in *Proceedings of the 31st USENIX Security Symposium*, 2022, pp. 1415–1432. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity22/presentation/nguyen>
- [10] P. Rieger, T. Krauß, M. Miettinen, A. Dmitrienko, and A. R. Sadeghi, “CrowdGuard: Federated backdoor detection in federated learning,” in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2024.
- [11] H. Yang, W. Xi, Y. Shen, C. Wu, and J. Zhao, “RoseAgg: Robust defense against targeted collusion attacks in federated learning,” *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 2951–2966, 2024.
- [12] Y. Chen, F. Luo, T. Li, T. Xiang, Z. Liu, and J. Li, “A training-integrity privacy-preserving federated learning scheme with trusted execution environment,” *Information Sciences*, vol. 522, pp. 69–79, 2020.
- [13] T. Xu, K. Zhu, A. Andrzejak, and L. Zhang, “Distributed learning in trusted execution environment: A case study of federated learning in SGX,” in *Proceedings of the 7th IEEE International Conference on Network Intelligence and Digital Content (IC-NIDC)*, 2021, pp. 450–454.
- [14] S. Queyruat, V. Schiavoni, and P. Felber, “Mitigating adversarial attacks in federated learning with trusted execution environments,” in *Proceedings of the IEEE 43rd International Conference on Distributed Computing Systems (ICDCS)*, 2023, pp. 626–637.
- [15] Z. Lu, L. Wang, Z. Zhang, M. Huang, J. Wang, and M. Li, “TMT-FL: Enabling trustworthy model training of federated learning with malicious participants,” *IEEE Transactions on Dependable and Secure Computing*, vol. 22, no. 3, pp. 2723–2740, 2025.
- [16] E. Bagdasaryan, A. Veit, Y. Hua, D. Estrin, and V. Shmatikov, “How To Backdoor Federated Learning,” in *Proceedings of the Twenty Third International Conference on Artificial Intelligence and Statistics*, vol. 108, 2020, pp. 2938–2948. [Online]. Available: <https://proceedings.mlr.press/v108/bagdasaryan20a.html>
- [17] B. Wang, Y. Yao, S. Shan, H. Li, B. Viswanath, H. Zheng, and B. Y. Zhao, “Neural Cleanse: Identifying and mitigating backdoor attacks in neural networks,” in *Proceedings of the IEEE Symposium on Security and Privacy (SP)*, 2019, pp. 707–723.
- [18] J. Zhang, C. Zhu, X. Sun, C. Ge, B. Chen, W. Susilo, and S. Yu, “FLPurifier: Backdoor defense in federated learning via decoupled contrastive training,” *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 4752–4766, 2024.
- [19] Z. Ma, J. Ma, Y. Miao, Y. Li, and R. H. Deng, “ShieldFL: Mitigating model poisoning attacks in privacy-preserving federated learning,” *IEEE Transactions on Information Forensics and Security*, vol. 17, pp. 1639–1654, 2022.
- [20] L. Wang, M. Huang, Z. Zhang, M. Li, J. Wang, and K. Gai, “RaSA: Robust and adaptive secure aggregation for edge-assisted hierarchical federated learning,” *IEEE Transactions on Information Forensics and Security*, vol. 20, pp. 4280–4295, 2025.
- [21] X. Zhang, Z. Chen, G. Chen, X. Feng, Q. Shen, and Z. Wu, “RPPFL: Robust and privacy-preserving federated learning via trusted execution environments,” in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2025, pp. 1–5.